BLOCK 3 · KNOW WHERE YOU'RE GOING

Path Planning & Navigation

Lab 5 — Autonomous Warehouse Robot Project

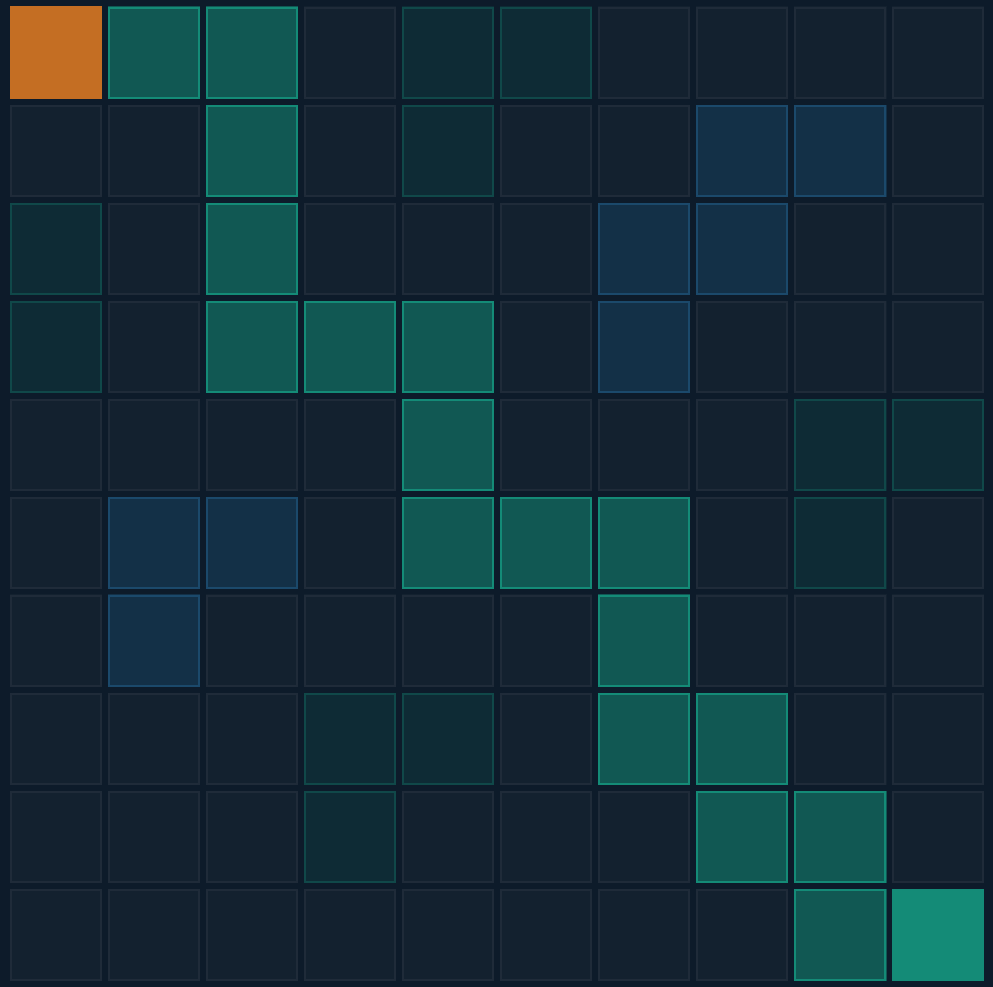
INTRODUCTION TO ROBOTICS FOR AI & DATA SCIENCE

Dr. Akram Fatayer

a.fatayer@zuj.edu.jo

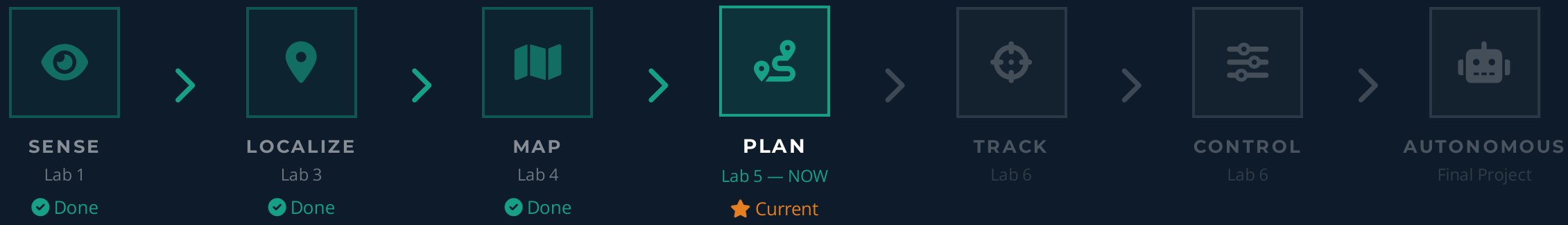
linkedin.com/in/akram-fatayer

Al-Zaytoonah University of Jordan



A* PATH PLANNING VISUALIZATION

Integration Map: You Are Here



In Lab 3 (Localization) your robot learned **where it is**. In Lab 4 (Mapping) it built a map of the environment. Now in Lab 5, it must decide **how to get somewhere** — path planning is the bridge that connects all three pillars of autonomous navigation.

The Path Planning Problem

Configuration Space (C)

All possible robot states — positions, orientations, joint angles. For a 2D mobile robot: (x, y, θ)

Free Space (C_{free})

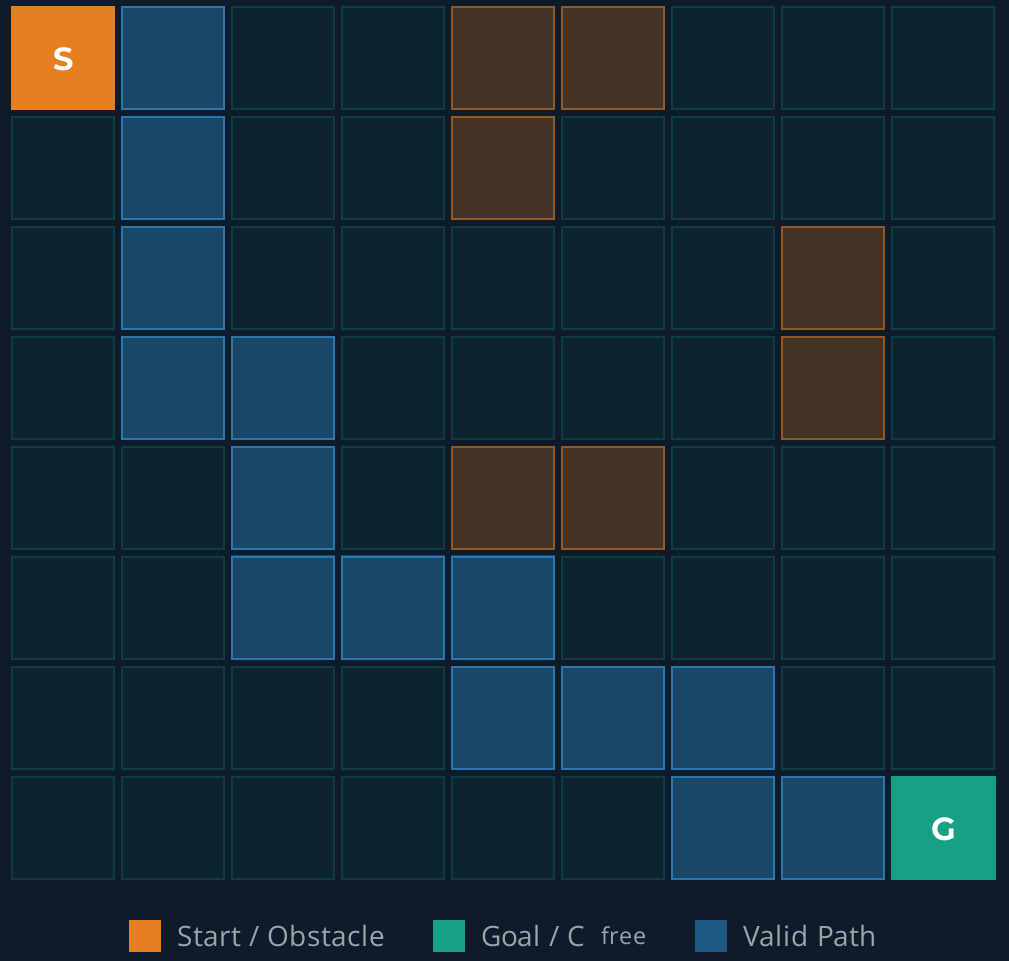
Configurations where the robot does **not** collide with any obstacle. The robot can safely occupy these states.

Obstacle Space (C_{obs})

Configurations that result in collision. The robot must **never** enter these states.

Valid Path π

A continuous sequence from q_{start} to q_{goal} that lies entirely within C_{free}. The planner's job is to find this path.



Three Approaches to Path Planning

Property	Graph-Based (A*, Dijkstra)	Sampling-Based (RRT, RRT*)	Potential Fields
Completeness	Complete Always finds solution if one exists	Probabilistically Complete Finds solution given enough time	Not Complete Can get stuck in local minima
Optimality	Optimal Finds shortest path (with admissible heuristic)	Not Optimal RRT* variant is asymptotically optimal	Not Optimal Path depends entirely on gradient
Computation Speed	Fast to Medium Scales poorly in high dimensions	Fast Highly efficient in complex, high-D spaces	Very Fast Real-time computation (10-100Hz)
Best Use Cases	2D/3D grids, known maps, when optimal path is required	High-dimensional spaces (robot arms), complex obstacles	Real-time reactive navigation, dynamic obstacle avoidance

A*: The Workhorse of Robot Planning

$$f(n) = g(n) + h(n)$$

TOTAL ESTIMATED COST

g(n): Actual Cost

The exact, known cost of the path from the **start node** to the current node *n*.

h(n): Heuristic Estimate

The estimated cost from node *n* to the **goal**. This is the "smart" part that guides the search.

Admissible Heuristics: Must *never overestimate* the true cost.

- **Euclidean:** Straight-line distance (continuous space).
- **Manhattan:** Grid distance (4-way movement).

Complete & Optimal

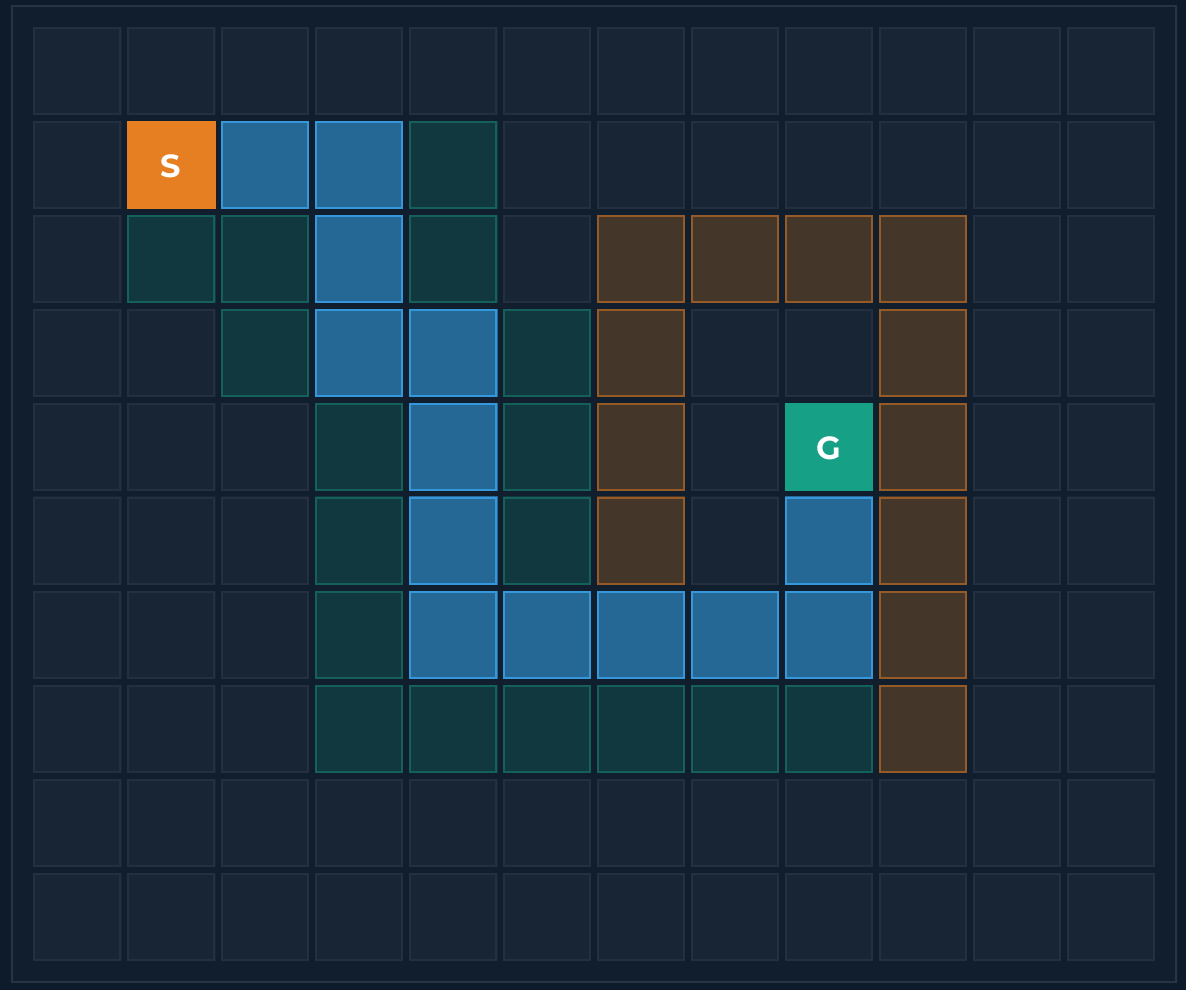
Complete: If a valid path exists, A* is guaranteed to find it. If no path exists, it will report failure.

Optimal: As long as the heuristic *h(n)* is admissible, A* is guaranteed to find the **shortest possible path**.

Highly Efficient

Unlike Dijkstra's algorithm which explores uniformly in all directions, A* uses the heuristic to **focus exploration toward the goal**. This drastically reduces the number of nodes evaluated, making it fast enough for real-time grid navigation.

A* in Action: Grid Search Visualization



PATH LENGTH

18

cells from start to goal

NODES EXPLORED

24

out of 120 total cells

Why A* is Efficient

A* explored only **20%** of the grid (24/120 cells). It uses the heuristic $h(n)$ to actively guide the search *toward* the goal, ignoring unpromising directions.

In contrast, **Dijkstra's Algorithm** would explore uniformly in all directions (like a growing circle), exploring roughly **60-80%** of the cells before finding the goal.

RRT: Planning in Continuous Space

The RRT Algorithm

1 Sample

Pick a random point in the configuration space `q_rand`.

2 Nearest

Find the closest existing node in the tree `q_near`.

3 Steer

Move a small distance from `q_near` toward `q_rand` to create a new node `q_new`.

4 Check Collision

Ensure the path between `q_near` and `q_new` is in free space.

5 Add to Tree

If collision-free, add `q_new` to the tree. Repeat until goal is reached.

Probabilistic Completeness

Unlike A*, RRT uses random sampling. It is guaranteed to find a solution **if one exists**, provided it runs for an infinite amount of time.

Continuous Space

RRT operates directly in continuous coordinates. It does **not require discretizing** the environment into a grid.

When to use RRT vs A*?

Use **A*** for 2D/3D grids (like mobile robots).

Use **RRT** for **high-dimensional spaces** (like 6-DOF robot arms) where creating a grid would require impossible amounts of memory.

Potential Fields: Reactive Navigation

The Physics of Navigation

Attractive Force

The goal acts like a magnet, creating a potential field that constantly **pulls the robot toward it**.

Repulsive Force

Obstacles act like opposing magnets, creating fields that **push the robot away** to prevent collisions.

Gradient Descent

The robot simply follows the gradient of the **sum of all forces**, moving downhill toward the goal.

$$U(q) = U_{att}(q) + U_{rep}(q)$$

The Achilles' Heel

The Local Minima Problem

Because the robot only looks at the **immediate local forces**, it can get trapped in locations where the attractive and repulsive forces perfectly cancel each other out.

- **Total Force is Zero:** The robot stops moving, believing it has reached the "bottom" of the potential field.
- **U-Shaped Obstacles:** The most common trap. The goal pulls the robot into the "U", but the walls push back equally.
- **The Consequence:** Pure potential fields are fast and reactive, but **cannot guarantee** finding a path in complex environments.

Hybrid Architecture: Global + Local Planning

Three-Layer Architecture

1. Global Planner (Strategic)

RUNS ONCE

A* or RRT plans a collision-free path from start to goal using the known static map. Output is a sequence of **waypoints**.

2. Local Planner (Tactical)

10 - 100 Hz

Potential Fields follows the global waypoints while reacting to **dynamic obstacles** (people, other robots). Output is **velocity commands**.

3. Controller (Reactive)

100 - 1000 Hz

PID or MPC converts velocity commands into actual **motor voltages**, handling low-level hardware constraints.

Separation of Concerns

Each layer handles different **time scales** and **spatial scales**. The global planner doesn't worry about robot dynamics; the controller doesn't worry about long-term goals.

Robustness

If the local planner encounters a blocked path, it triggers the global planner to **replan**. If a new obstacle appears, the local planner **adapts instantly** without a full replan.

Real-World Examples

- **ROS Navigation Stack:** Uses A* (global) + Dynamic Window Approach (local).
- **Autonomous Cars:** Uses A* for route planning (global) + Model Predictive Control for lane keeping and obstacle avoidance (local).

Choosing the Right Algorithm

A* Search

GRAPH-BASED

ENVIRONMENT

2D/3D Grids and known, static maps.

ROBOT TYPE

Mobile robots, warehouse AGVs, and game characters.

KEY STRENGTH

Guarantees the **optimal shortest path** when using an admissible heuristic.

RRT

SAMPLING-BASED

ENVIRONMENT

Continuous Space with complex geometric obstacles.

ROBOT TYPE

Robot arms (6+ DOF), humanoids, and complex kinematics.

KEY STRENGTH

Highly efficient in **high-dimensional spaces** where grids fail.

Potential Fields

REACTIVE CONTROL

ENVIRONMENT

Dynamic, unknown, or rapidly changing environments.

ROBOT TYPE

Drones, fast-moving vehicles, reactive obstacle avoidance.

KEY STRENGTH

Real-time computation (10-100Hz) for immediate reaction to sensor data.

Key Takeaways

A* is the Workhorse

For 2D/3D grid environments, A* is the optimal and complete solution. It uses heuristics to efficiently guide the search toward the goal.

Planning Requires a Map

You cannot plan a path without knowing the **Configuration Space (C-space)**. This relies on Localization (Lab 3) and the Occupancy Grid Map (Lab 4). Path planning consumes the map as input.

Hybrid Architecture is Standard

Real-world robots use a **Global Planner** (like A*) for the strategic route, and a **Local Planner** (like Potential Fields) for real-time obstacle avoidance.

Next Up: Lab 5 (Tracking)

We now have a planned path. Next, we must figure out how to generate the exact motor commands to make the robot's wheels **follow that path accurately**.

Questions & Discussion

Thank you for your attention.

Dr. Akram Fatayer

a.fatayer@zuj.edu.jo

linkedin.com/in/akram-fatayer

Al-Zaytoonah University of Jordan